

Content Agent

Production AI System Case Study

An end-to-end research and publishing workflow that retrieves evidence, drafts content, verifies claims, routes work through two human approval gates, and publishes to a live website.

100%

Recall@3 on a 30-query
golden set

96.7%

Recall@1 on the same
evaluation set

62

Mocked regression tests

What the system does

A user enters a topic. The system searches the web and a local knowledge base, drafts an article from the retrieved evidence, checks factual claims against sources, reflects on the result, and either revises or moves forward. Nothing is published until a person approves both the content and the final page layout.

Live case study: themachinist.org/content-agent

Portfolio: themachinist.org

System flow

Retrieve	→	Draft	→	Verify	→	Reflect	→	Revise / Continue	→	Human Gate 1	→	Render HTML	→	Human Gate 2	→	Publish
----------	---	-------	---	--------	---	---------	---	----------------------	---	-----------------	---	----------------	---	-----------------	---	---------

Core architecture

Retrieval	Tavily web search + Qdrant local knowledge base; BM25 and dense retrieval fused with Reciprocal Rank Fusion.
Reasoning	DeepSeek API for drafting, claim verification, reflection, and feedback-driven revision.
Workflow	LangGraph state machine with bounded retries, conditional routing, and durable checkpoints.
Backend	FastAPI with SqliteSaver for interruption-safe human review and workflow resumption.
Deployment	Docker on AWS EC2, Caddy for HTTPS, Qdrant on a private network, and Netlify for the published site.

Reliability, evaluation, and control

The project was designed around measurable behavior and safe operation. The model is only one component; the surrounding system handles evidence quality, failures, approvals, publishing safeguards, and reproducibility.

Evaluation	Golden retrieval set, claim-level grounding checks, pre-registered experiments, and a fixed verifier fixture in CI.
Human control	Two separate approval gates: one for factual content and one for layout. The second gate cannot alter approved text.
Failure handling	Timeouts, retries, invalid-key tests, bounded revision loops, cost controls, and rollback protection.
Observability	Structured logs, latency and cost telemetry, prompt-content hashes, and source-chunk attribution for verified claims.
Deployment safety	Non-root container, private vector database, HTTPS, CI gates, fork validation, and deliberate publish action.

Key engineering decisions

- One agent, not many: A single coordinated graph kept state, debugging, and failure handling simpler.
- Feedback-driven revision: Failed claims are returned to the next draft as specific corrections; blind re-drafting was removed after testing.
- Two revisions maximum: A third pass did not improve grounded substance and sometimes made claims vaguer, so the limit stayed at two.
- Quality measured by substance: The primary metric rewards verified substantive claims, with unverified claims capped at 15 percent.
- Publishing separated from generation: The agent can prepare and stage work, but a person controls the final release.

Selected findings

Retrieval freshness mattered more than query rewriting	Weak grounding on technical topics was traced to stale or empty cached retrieval. Fresh evidence recovered roughly 90 percent of formula claims.
The verifier was not the bottleneck	Independent adjudication confirmed all 27 substantive claims in a healthy sample were classified correctly.
Plausible telemetry can still be wrong	A chained assignment silently wrote one timing value into another field. Splitting the metrics exposed the issue.
Rollback code is production code	An unsafe force-delete path was caught before release and replaced with a history-preserving revert.

Production readiness and business value

The system turns a broad content request into a controlled production workflow with evidence, measurable quality, human accountability, and repeatable deployment. The same design patterns apply to enterprise research, document processing, knowledge workflows, and regulated review processes.

Reliable output	Every factual claim is checked against retrieved sources and can be traced to supporting chunks.
Operational continuity	Checkpointed state allows long-running work to pause for review and resume without losing progress.
Lower review risk	Content and layout approvals are separated, preventing a design change from silently modifying approved facts.
Controlled iteration	Evaluation and regression tests identify whether changes improve retrieval, grounding, or workflow behavior before release.
Cost awareness	Per-run telemetry tracks model calls, retries, latency, and cost so improvements can be measured against operating constraints.

Technology stack

AI and orchestration	LangGraph, DeepSeek, tool calling, structured outputs, human-in-the-loop workflows
Retrieval	Qdrant, BM25, dense embeddings, Reciprocal Rank Fusion, Tavily
Backend and state	Python, FastAPI, SqliteSaver, GitPython
Infrastructure	Docker, AWS EC2, Caddy, Netlify, private container networking
Quality	Golden datasets, regression tests, failure injection, CI gates, telemetry, rollback testing

Known limitation: workflow state survives a restart, but the API's in-memory map of active runs is not yet rebuilt automatically. The limitation is documented, and the next step is to restore active-run discovery from persisted checkpoints.

Project links

Full case study and live interface

<https://themachinist.org/content-agent>

Portfolio

<https://themachinist.org>

GitHub

<https://github.com/anudeepreddy332>

Author: Anudeep Reddy Mutyala

AI / ML Engineer - Agentic Systems, Applied ML, and Production AI Workflows